

Lucianu Dragos - Adrian

1. noduri critice : 2, 3, 4, 5, 6

2. ~~2~~ noduri critice : noduri pe care dacă le eliminăm alături de muchiile lor graful nu mai este conex

2. muchii critice : (2, 3); (3, 4); (3, 6)

muchii critice : muchii pe care dacă le eliminăm graful nu mai este conex

3. $df(3)$:

plecăm din nodul 3

ne uităm la vecinii nodului 3

mergem în nodul 2

ne uităm la vecinii nodului 2

mergem în nodul 1

ne uităm la vecinii nodului 1

mergem în nodul 9

ne uităm la vecinii săi

se întorcem în nodul 1

se întorcem în nodul 2

se întorcem în nodul 3

mergem în nodul 4

ne uităm la vecinii săi

mergem în nodul 5

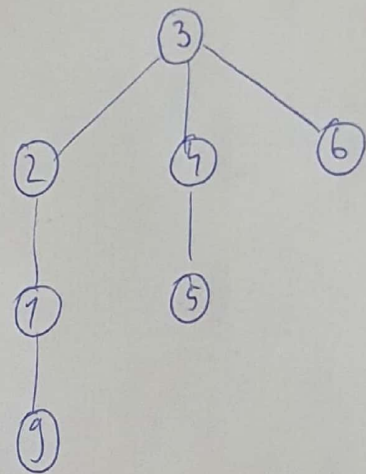
ne uităm la vecinii săi

se întorcem în nodul 4

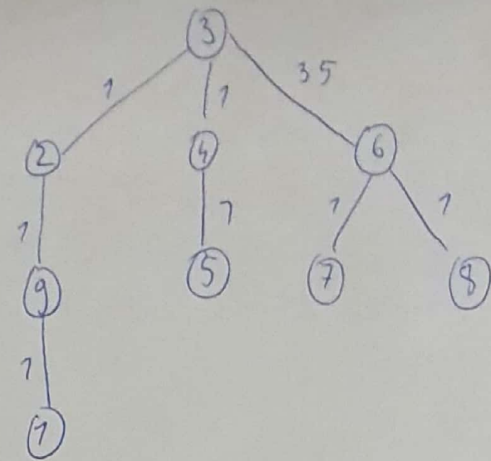
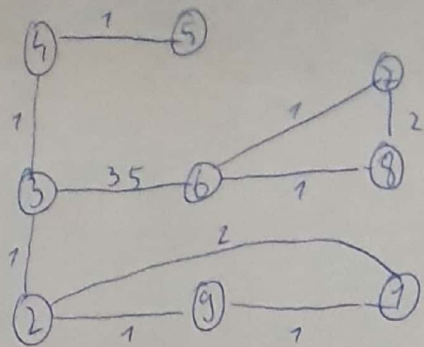
se întorcem în nodul 3

mergem în nodul 6

am vizitat 7 noduri



4.



5.

	0	n 1	e 2	r 3	t 4	a 5	n 6	t 7	a 8
0	0	1	2	3	4	5	6	7	8
e 1	1	1	1	2	3	4	5	6	7
n 2	2	2	2	2	3	4	5	6	7
a 3	3	3	3	3	3	3	4	5	6
n 4	4	4	4	4	4	4	4	5	6
e 5	5	5	5	4	5	5	5	5	6
n 6	6	6	5	5	6	6	5	6	6

distanța de editare este matrice $[6][8] = 6$

Gincianu Dragos - Adrian

- b. Aceasta problemă se bazează pe sortare topologică, unde n reprezintă nr de noduri, iar perechile (i, j) reprezintă muchii direcționate de la i la j , iar n reprezintă nodul unde dorim să ajungem (oprim algoritmul).

Pentru primul algoritm luăm într-o coadă toate v cu grad intern 0, adăugăm în sortare, luăm toate muchiile care scade gradul ^{intern} nodurilor adiacente și de acesta are grad intern 0 îl adăugăm la coadă : Algoritm :

coadă c

adăugăm în c nodurile i cu $\text{deg}[i] = 0$

cât timp $c \neq \emptyset$

$i \leftarrow$ luăm din coadă

adăugăm în sortare

pentru $i, j \in E$ executa

$$d[j] = d[i] - 1$$

dacă $d[j] = 0$ atunci

adăugăm j în c

Al doilea algoritm :

Realizăm lașat pe DF și realizăm încă un vector final $[v]$ = momentul la care a fost finalizat v în DFS, în muchia $i, v \in E \Rightarrow \text{val}[v] > \text{val}[i]$

sortarea topologică este sortarea descrescătoare la final

iar la nodul n se oprim

Intare topologică (graf):

1. ciclism $O(FC \text{ graf})$ nt a calc timpii final $\phi(v)$ nt fiecare v
2. când fiecare ^{nod} este finalizat, inserăm în capul listei întârziere
3. returnăm lista întârziată de ~~nt~~ noduri

complexitate : $DF \Rightarrow O(n + m)$, unde care de se
este la finalul DF -ului